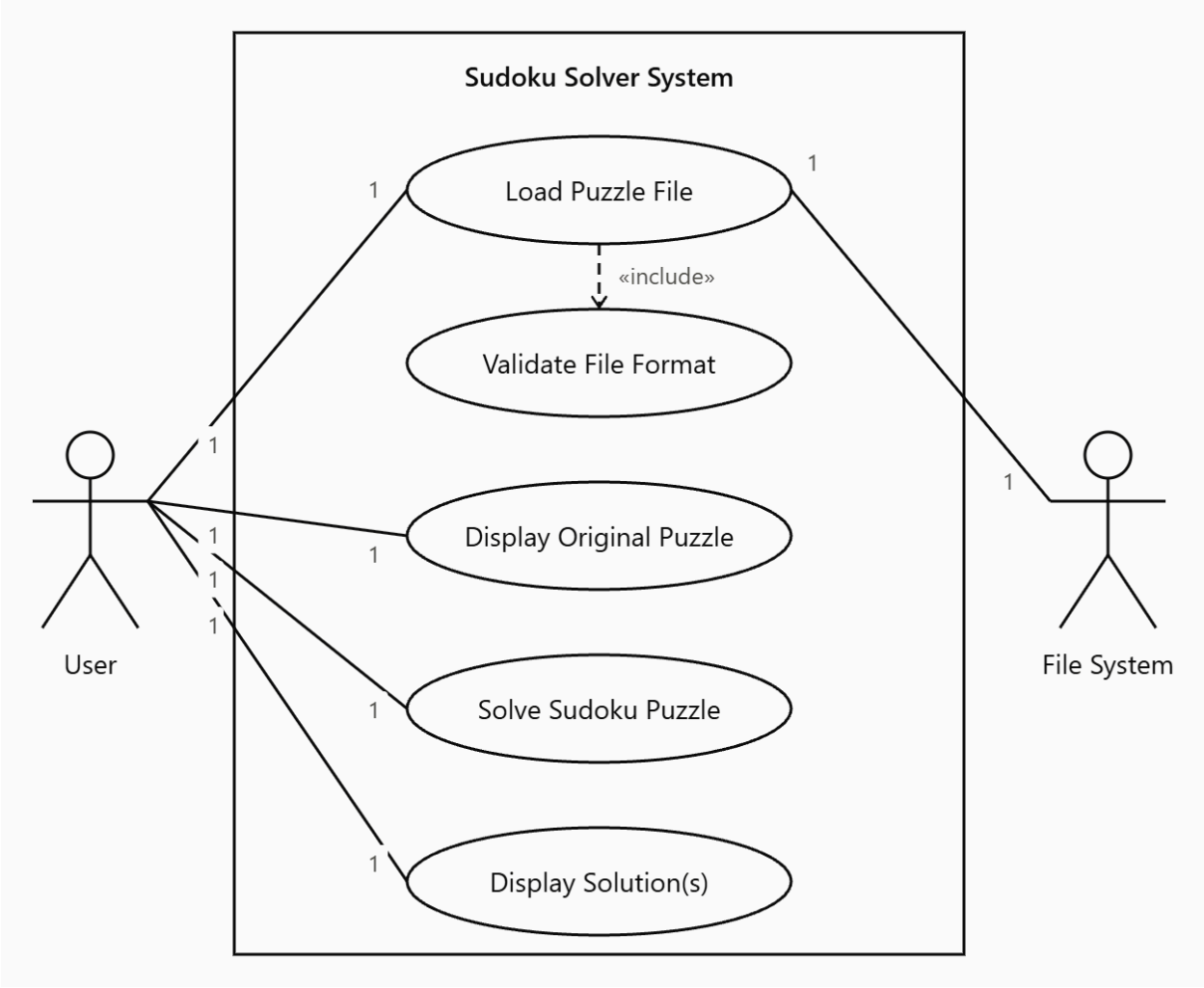


a. Identify Use Cases
   i. Load Puzzle File
   ii. Validate File Format
   iii. Display Original Puzzle
   iv. Solve Sudoku Puzzle
   v. Display Solution(s)

b. Brief Descriptions
   i. **Load Puzzle File**: Allows the user to trigger the loading of raw puzzle text data from the external file system into the application memory.
   ii. **Validate File Format**: A mandatory included use case that checks the integrity, structure, and character format of the loaded puzzle file data.
   iii. **Display Original Puzzle**: Presents the initial unsolved 9x9 grid configuration to the terminal.
   iv. **Solve Sudoku Puzzle**: Executes the recursive depth-first search with backtracking to evaluate cell constraints and find valid solutions.

v. **Display Solution(s)**: Outputs all valid completed solution grids or indicates if no solution exists.

c. Flow of Events

1. User requests to load a puzzle file.
2. System retrieves raw data from the File System.
3. System automatically triggers file format validation (<<include>>).
4. System displays the loaded original puzzle.
5. User initiates the puzzle solving routine.
6. System runs recursive backtracking to find solutions.
7. System displays the final solution(s) to the user.

d. Detailed Description

Use Case: Solve Sudoku Puzzle

1. User Initiates Puzzle Solving

- The user executes the program manager to begin processing the loaded puzzle files.
- The system receives the command, retrieves the loaded 9x9 board configuration from memory, and initializes the recursive solver component.

2. System Scans for Empty Cells

- The system searches the grid to locate the next available empty cell marked by an underscore (_).
- If the system finds no empty cells, it registers a complete grid configuration, stores it in the solutions list, and returns to previous states.

3. System Tests Candidate Digits

- If an empty cell is present, the system loops through potential candidate digits from '1' to '9'.
- The system evaluates each candidate using row, column, and 3x3 block validation rules to ensure compliance with Sudoku constraints.

4. System Executes Recursive Backtracking

- Upon finding a valid move, the system temporarily assigns the digit to the cell and recursively calls the solver function for the next step.
- If a dead-end or rule violation occurs down the path, the system performs a backtracking step, undoes the prior choice by resetting the cell to an underscore (_), and attempts the next available digit.

5. System Presents Results

- Following the exhaustive search, the system concludes the process by outputting the results to the user interface, printing every valid solution found or displaying a "No solution found" message if the configuration is invalid.